

Sistemas Operacionais

Processos e Threads

Conteúdo

- Processos
- Threads
- InterProcess Communication
- Algoritmos de escalonamento

PROCESSOS

- Processo é nome que se dá a um programa em execução
- Um processo controla os valores atuais dos registradores da CPU, bem como variáveis em memória
- Cada processo possui sua própria “CPU virtual”
- Em sistemas multiprogramáveis, os processos dividem a atenção da CPU
- Processos são comutados (chaveados) a fim de criar a ilusão de que cada usuário tem a CPU exclusivamente para si

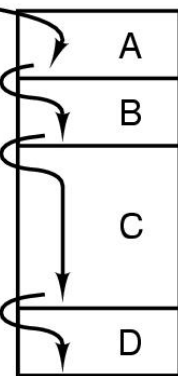
Processos (cont.)

- Exemplo de 4 processos competindo pela CPU
- Somente um processo por vez, pois só há uma CPU e assim só um registrador PC real

Cada processo possui seu próprio
“registrador PC lógico”

Chavamento de contexto

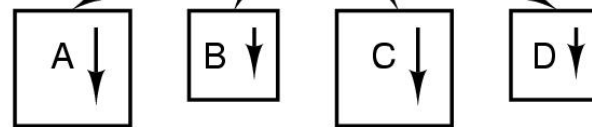
One program counter



(a)

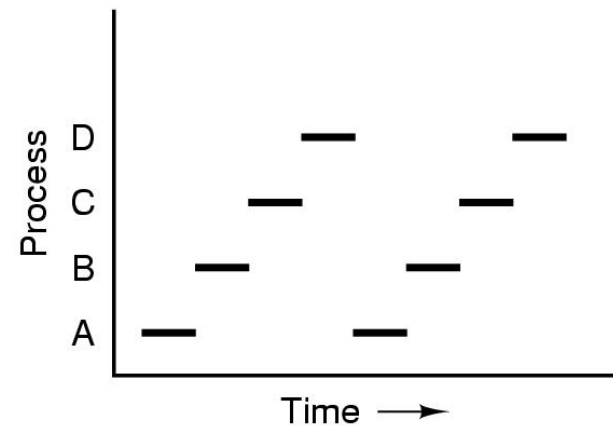
Process switch

Four program counters



(b)

Ilusão de que os 4 processos
estão em paralelo



(c)

Criação de um Processo

Processos são criados por 4 eventos principais:

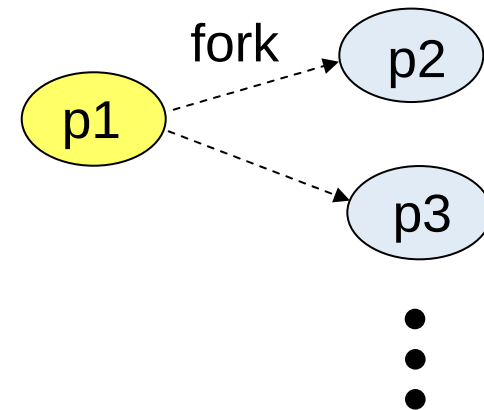
- Inicialização do sistema
 - Processos *foreground* e *background* (*daemons*)
- Processos em execução que usam uma “*system call*” para criar outros processos
 - Ex.: leitura de grandes quantidades de dados via um *buffer*
- Solicitação de usuário para criação de um novo processo
 - Ex.: disparo de novos processos via teclado ou mouse
- Inicialização de um *job* em sistemas *batch*
 - Para cada job um processo é criado pelo processo SO

Criação de um Processo (cont.)

- Processos são sempre criados por um processo em execução via a “system call” correspondente

- *System call:*

- No unix → fork
- No windows → CreateProcess



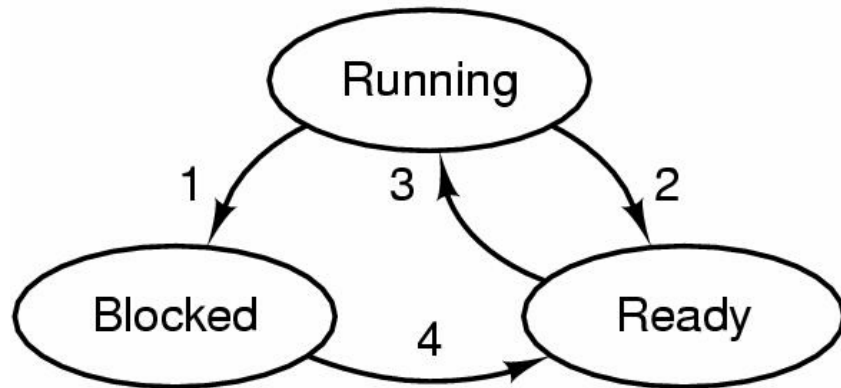
- Quando criado, um processo filho tem o seu próprio espaço de endereçamento
 - No unix, isso ocorre em duas etapas

Destruição de um Processo

Há 4 situações em que um processo é terminado

- Finalização normal ao término da sua tarefa (**voluntário**)
 - O próprio processo fecha seus arquivos abertos, e termina
- Finalização por erro detectado durante a sua execução (**voluntário**)
 - Processo detecta um erro externo e aborta execução
- Finalização por erro fatal (**involuntariamente**)
 - Processo contém um erro (bug)
- Finalização por outro processo (**involuntariamente**)
 - O outro processo tem que ter privilégio para tal

Estados de um Processo



1. Process blocks for input
2. Scheduler picks another process
3. Scheduler picks this process
4. Input becomes available

Um processo pode se encontrar nos seguintes estados

- Execução (running)
 - Processo ocupa a CPU (exclusivamente)
- Pronto (ready)
 - Processo está pronto para ocupar a CPU, mas deve esperar pela sua vez em fila
- Bloqueado (blocked)
 - Processo não pode executar enquanto um evento externo (por exemplo, a resposta do controlador de HD) não ocorrer

Informações agregadas a um Processo

- O SO possui uma tabela contendo o ID de cada processo
- Cada registro da tabela refere-se a um processo
- Tais registros são chamados de PCB (Process Control Block)

PCB1
PCB2
PCB3
⋮

← Tabela de processos

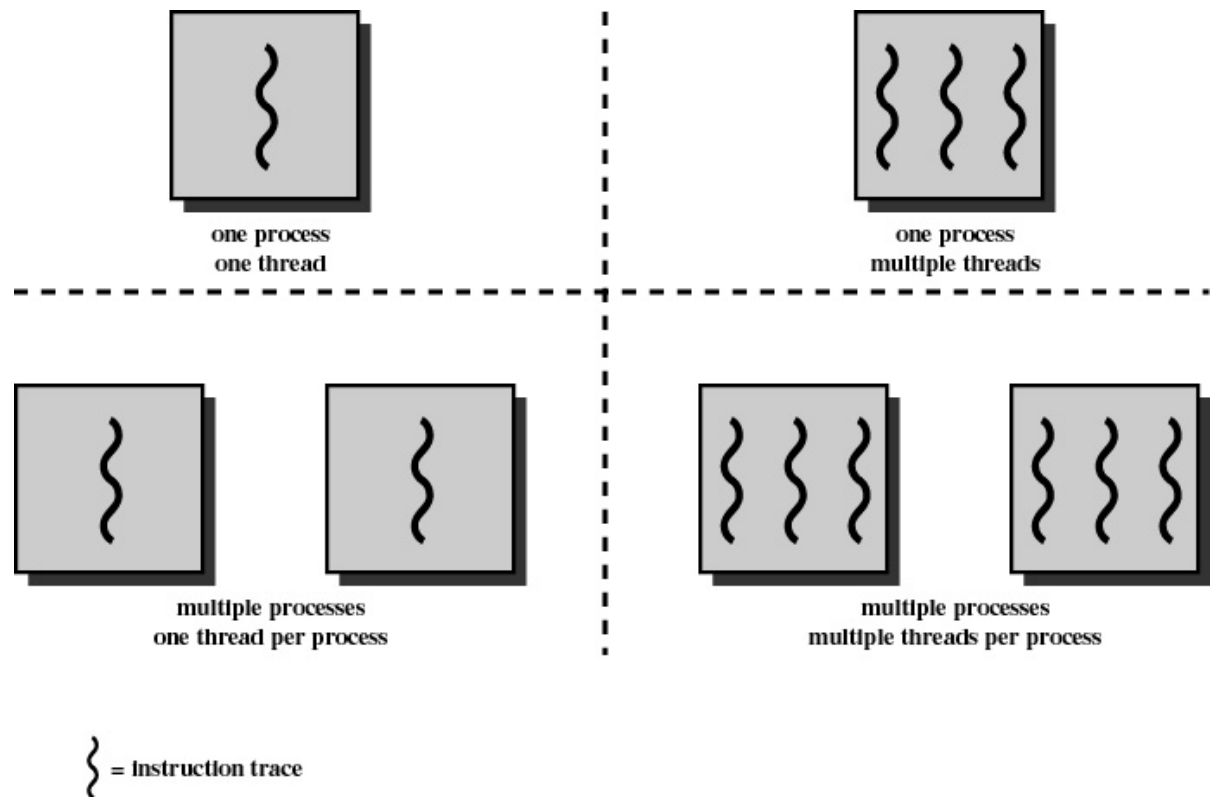
- Todas as informações relevantes aos processos estão nos seus PCBs. Ex.: o registrador PC, o registrador de pilha, os arquivos abertos, o usuário dono do processo, etc

THREADS

- As threads otimizam o desempenho dos processos
 - Várias tarefas podem ser executadas “em paralelo” dentro do processo
 - Chaveamento entre processos é muito dispendioso quando desnecessário

Nesse contexto,
os processos agrupam
recursos e as threads
executam as tarefas junto
à CPU

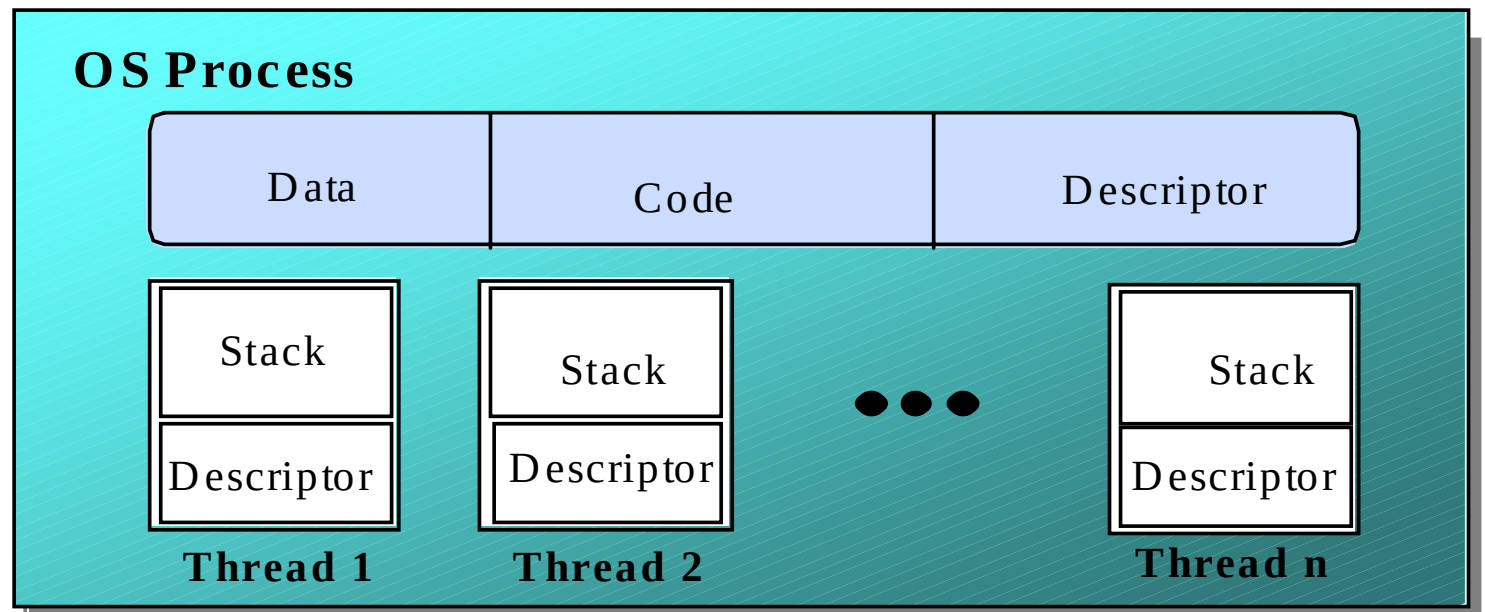
Uma thread representa
um fluxo de execução
dentro do processo



Multithreading

info compartilhada por
todas as threads

PROCESSO



chaveamento
entre threads
é muito mais
simples e
rápido!

as threads trabalham em cooperação e
pertencem a um mesmo usuário

Características das Threads

- Tal qual um processo, uma thread pode estar em um dos seguintes estados:
 - Execução
 - Pronto
 - Bloqueado
- Uma thread pode criar outras threads, mas o espaço de endereçamento é comum a todas
- Todas as threads de um processo podem acessar/alterar um arquivo ou alocar memória
- Cada thread tem a sua própria pilha para que o chaveamento entre threads ocorra eficientemente
- Como as threads são cooperativas, elas podem ceder a CPU umas às outras voluntariamente

Motivação para uso das Threads

- Muitas aplicações possuem múltiplas atividades que devem ser realizadas ao mesmo tempo
- Algumas dessas threads podem estar bloqueadas em alguns momentos
- Decompondo tais aplicações em múltiplas threads seqüenciais (rodando “em paralelo”) facilita a programação
- O fato de se compartilhar o espaço de endereçamento e todos os dados, torna as threads insubstituíveis
- A criação de uma thread pode ser até 100 vezes mais rápido do que a criação de um processo
- Threads são muito úteis em sistemas multiprocessados, onde o paralelismo real ocorre

Exemplo de uso das Threads

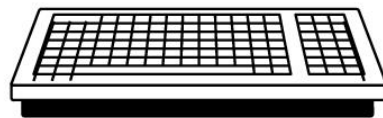
■ 1- PROCESSADOR DE TEXTO (EX.: WORD)

Cada thread trabalha independentemente

Controla formatação do texto na tela do micro

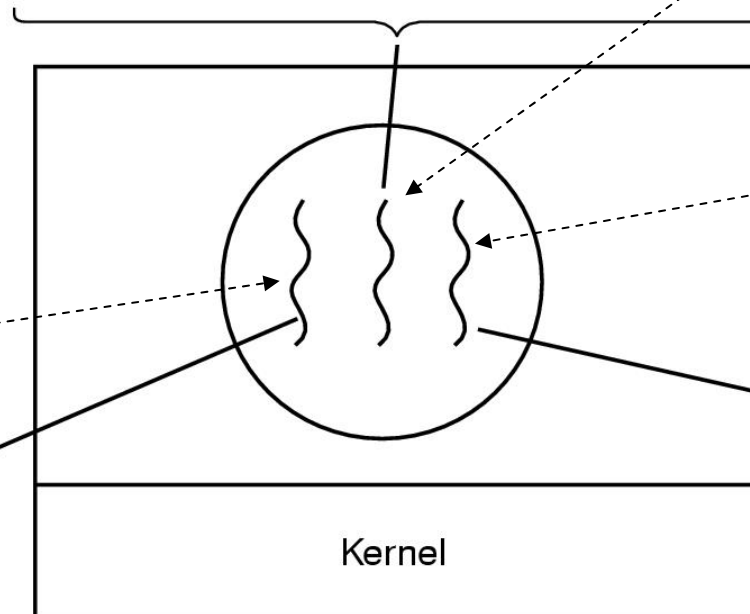
Usar 3 processos não funcionaria aqui: dados devem ser manipulados simultaneamente !

Controla info vinda do teclado

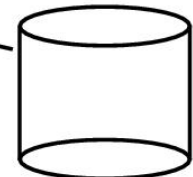


Keyboard

Four score and seven years ago, our fathers brought forth upon this continent a new nation: conceived in liberty, and dedicated to the proposition that all men are created equal.	Now we are engaged in a great civil war testing whether that nation, or any nation so conceived and so dedicated, can long endure. We are met on a great battlefield of that war.	We have come to dedicate a portion of that field as a final resting place for those who here gave their lives that this nation might live. It is altogether fitting and proper that we should do this.	But, in a larger sense, we cannot consecrate we cannot hallow this ground. The brave men, living and dead, who struggled here have consecrated it, far above our poor power to add or detract. The world will little note, nor long remember, what we say here, but it can never forget what they did here.	It is for us the living, rather, to be dedicated here to the unfinished work which they who fought here have thus far so nobly advanced. It is rather for us to be here dedicated to the great task remaining before us, that from these honored dead we take increased devotion to that cause for which they gave the last full measure of devotion, that we here highly resolve that these dead shall not have died in vain that this nation, under God, shall have a new birth of freedom and that government of the people, by the people, for the people, shall not perish from the earth.
--	---	--	---	---



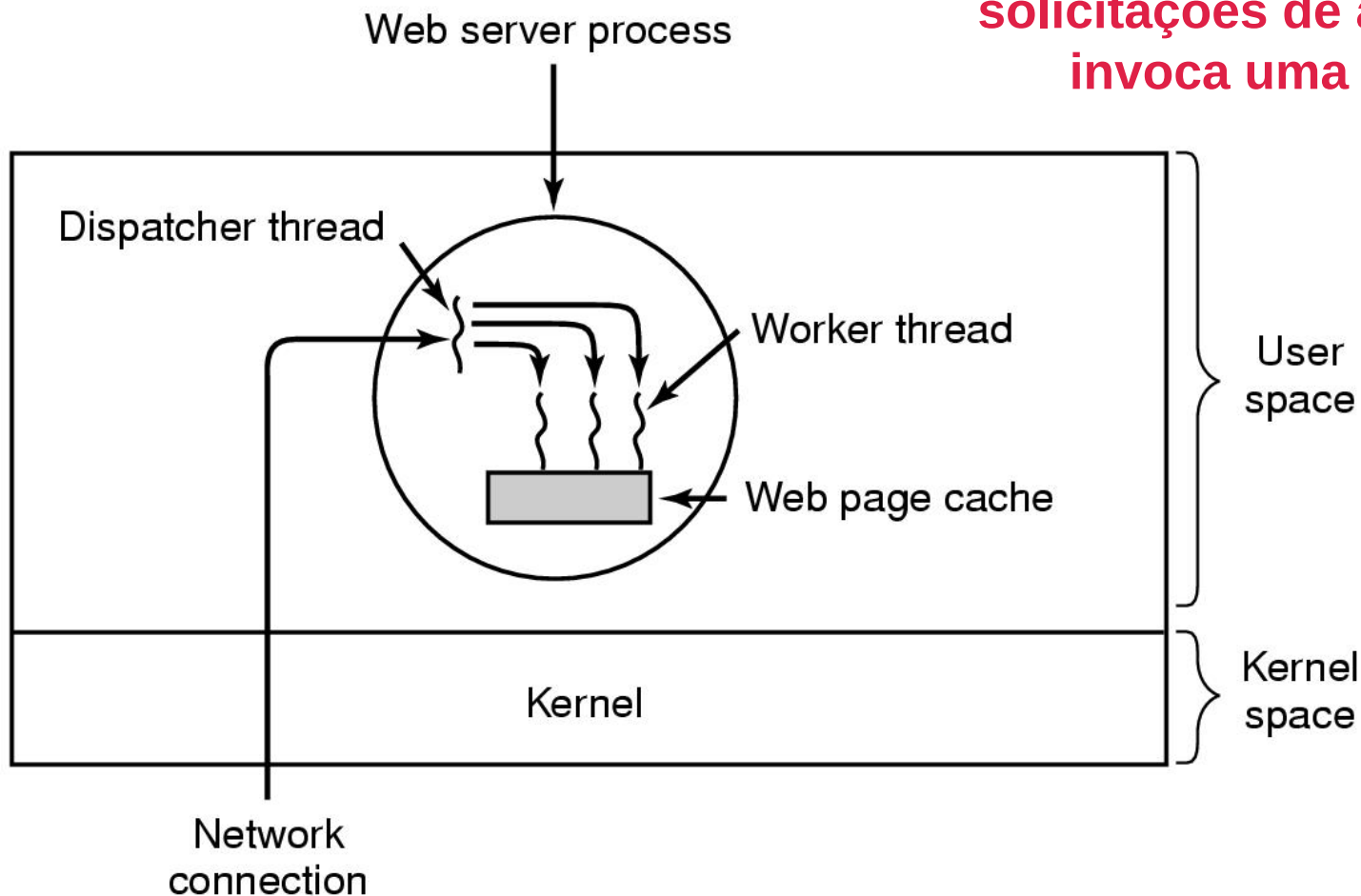
Controla backup automático



Disk

Outro exemplo de uso das Threads

■ 2- Um web server com multithread



A thread Dispatcher monitora solicitações de acesso à página e invoca uma thread worker

Usa página em cache se possível, **SENÃO** busca no Disco e fica bloqueado até recepção do disco



Possibilita ilusão de paralelismo

Implementação de Threads

- Threads podem ser implementadas como:
 - Biblioteca de pacotes em espaço de usuário
 - Pacote gerenciado pelo kernel
 - Pacote híbrido

IPC (InterProcess Communications)

- Processos normalmente se comunicam
 - Uma padronização para troca de info é necessária
 - Proteção entre os processos precisa ser garantida
 - Sincronização é fundamental
- Threads também se comunicam entre si, e as soluções são similares às apresentadas aqui p/ processos

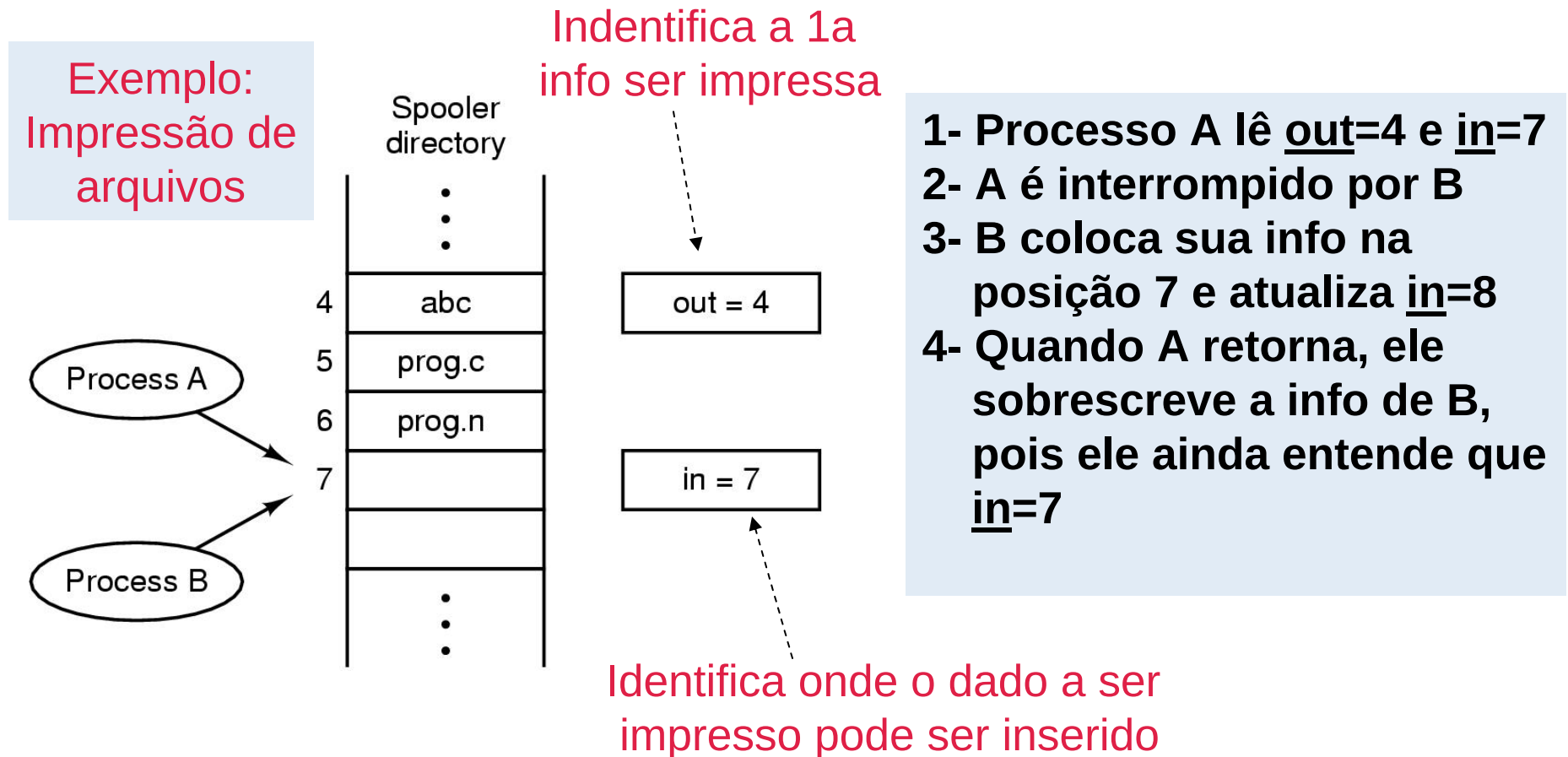
Processo A

Processo B



O Problema da Seção Crítica

- Quando dois ou mais processos compartilham um recurso, pode haver problemas, como ilustrado aqui:

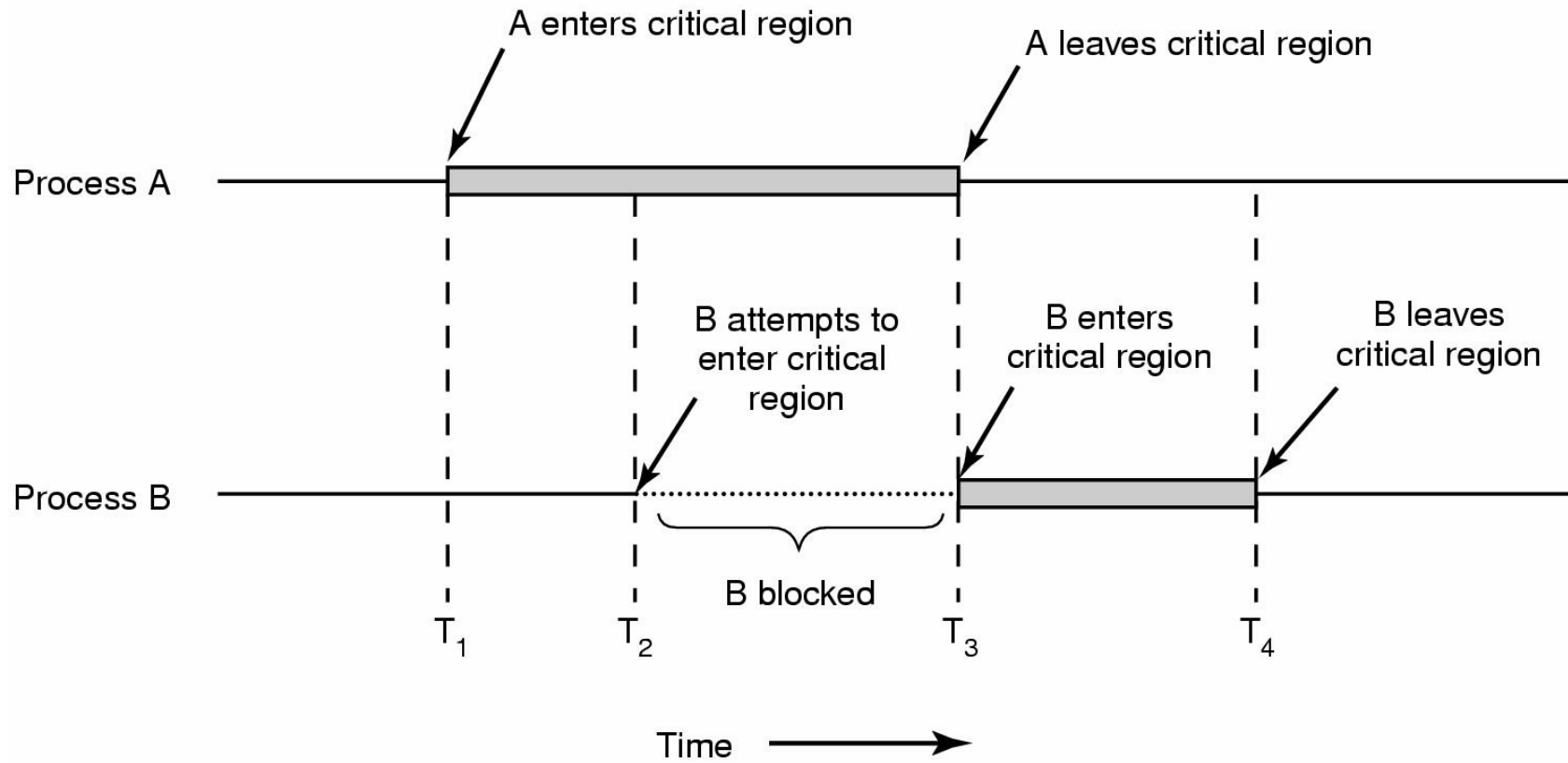


Solução ao Problema da Seção Crítica

- Evitar que mais de um processo acesse a seção crítica ao mesmo tempo
 - **EXCLUSÃO MÚTUA**
- Uma solução completa e eficiente deve satisfazer as 4 condições seguintes:
 - A seção crítica não deve jamais estar sendo acessada por dois processos simultaneamente
 - Nenhuma consideração a respeito da velocidade ou quantidade de CPUs deve existir
 - Nenhum processo rodando fora de sua seção crítica pode bloquear outros processos
 - Nenhum processo deveria esperar indefinidamente para acessar a sua seção crítica

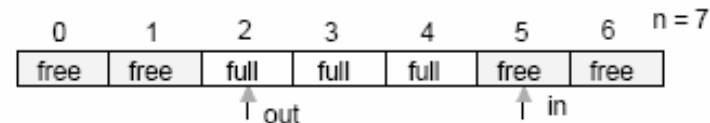
Solução ao Problema da Seção Crítica (cont.)

- A exclusão mútua evitando conflitos de acesso à seção (região) crítica



O Problema do produtor Consumidor

- Um produtor produz dados que são consumidos por um consumidor (ex.: spooler e impressora)
- Os dados produzidos e ainda não consumidos são armazenados em um BUFFER
- Podem haver vários produtores e consumidores
- Idealmente o BUFFER é de tamanho infinito



Variáveis:

“in” → onde dado dever ser Inserido

“out” → dado a ser retirado

```
var buffer: array[0..n-1] of items;    /* circular array */
in = 0
out = 0

/* producer */
repeat forever
  ...
  produce item nextp
  ...
  while (in+1 mod n == out)
    do nothing
  buffer[in] = nextp
  in = in+1 mod n
end repeat

/* consumer */
repeat forever
  while (in == out)
    do nothing
  nextc = buffer[out]
  out = out+1 mod n
  ...
  consume item nextc
  ...
end repeat
```

Algoritmos para Exclusão mútua

- Desabilitando interrupções
- Busy Waiting
 - Variável lock
 - Instrução Test and Set
- Semáforos
- Passagem de mensagens

Desabilitando interrupções

- Quando um processo entra na sua seção crítica, o mesmo desabilita todas as interrupções
- Fica assim garantido a esse processo acesso exclusivo à seção crítica
- Ao sair da seção crítica, as interrupções são habilitadas
- Problemas
 - Há que se permitir que processos de usuários desabilitem interrupções
 - Em sistemas multiprocessados (mais de uma CPU), somente uma CPU terá as interrupções suspensas, os processos nas demais CPUs podem acessar a seção crítica normalmente
- Somente viável para processos do sistema!

Busy Waiting – variáveis de trava (*lock*)

- Existe uma variável *lock* que é inicializada com zero
- Um processo desejando acessar a seção crítica verifica se *lock*=0
- Se sim, o processo entra na seção crítica e ajusta *lock*=1
- Demais processos devem aguardar até que *lock* retorne a zero
- Apresenta os mesmos problemas vistos no exemplo do servidor de impressão
- “*Busy waiting*” significa um processo executando um loop para detectar “seção crítica livre”
- ***Busy waiting*** → desperdício de CPU

Instrução TSL (hardware)

- TSL (Test and Set Lock) existe na maioria dos computadores

Ocorre busy waiting!

TSL RX, LOCK (RX ← LOCK e LOCK ← 1)

- TSL é indivisível, não pode ser interrompida

enter_region:

TSL REGISTER, LOCK	copy lock to register and set lock to 1
CMP REGISTER, #0	was lock zero?
JNE enter_region	if it was non zero, lock was set, so loop
RET	return to caller; critical region entered

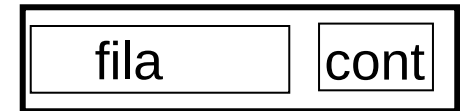
leave_region:

MOVE LOCK, #0	store a 0 in lock
RET	return to caller

Semáforo (S)

- Possui um contador (cont) e uma fila
- Somente duas operações: UP e DOWN

semáforo



```
DOWN(S){
    s.count-- ;
    if(s.count<0){
        coloque o processo na s.fila;
        entre em estado de bloqueio;
    } else {entra na seção crítica}
}
```

```
UP(S){
    s.count++ ;
    if(s.count<=0){
        remove um processo P da s.fila;
        coloque o processo P na lista de pronto;
    }
}
```

* Inicialmente s.cont=1,
a cada nova solicitação
pela seção crítica o seu
valor é decrementado

* O valor int. do contador
representa qtidade de
acessos pendentes

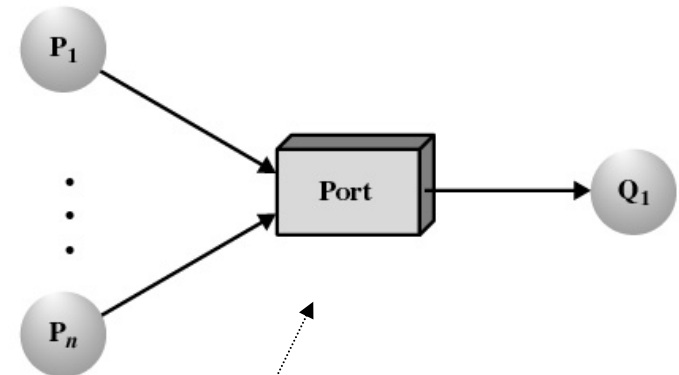
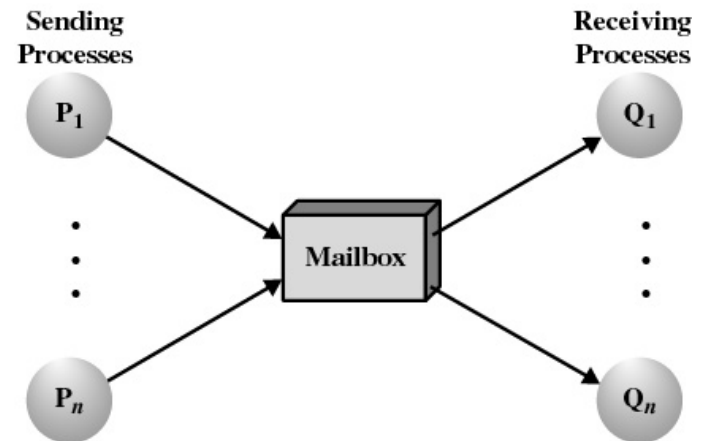
Ex.: após 3 solicitações
simultâneas

s.fila: 2 processos

s.count: -2

Passagem de Mensagens

- Mais apropriado à sistemas distribuídos e multiprocessados com memória compartilhada
- Mensagens
 - `Send(destino,mensagem)`
 - `Receive(origem,mensagem)`
- Perda de mensagens pode ocorrer
- Troca de mensagens pode ser
 - Direta
 - Indireta via caixas de mensagens ou portas



comum em arquiteturas
cliente servidor

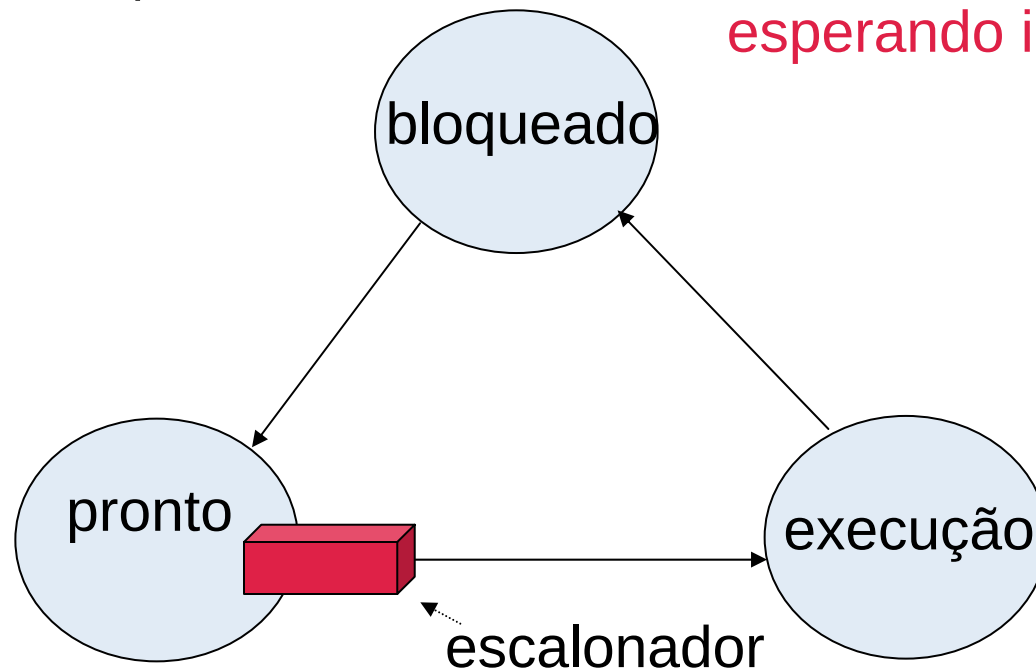
Algoritmos de Escalonamento

- Na multiprogramação, o escalonador decide qual processo deve obter a CPU

- Aspectos importantes
 - Tempo de resposta
 - Throughput
 - Eficiência do processador

Um bom escalonador !

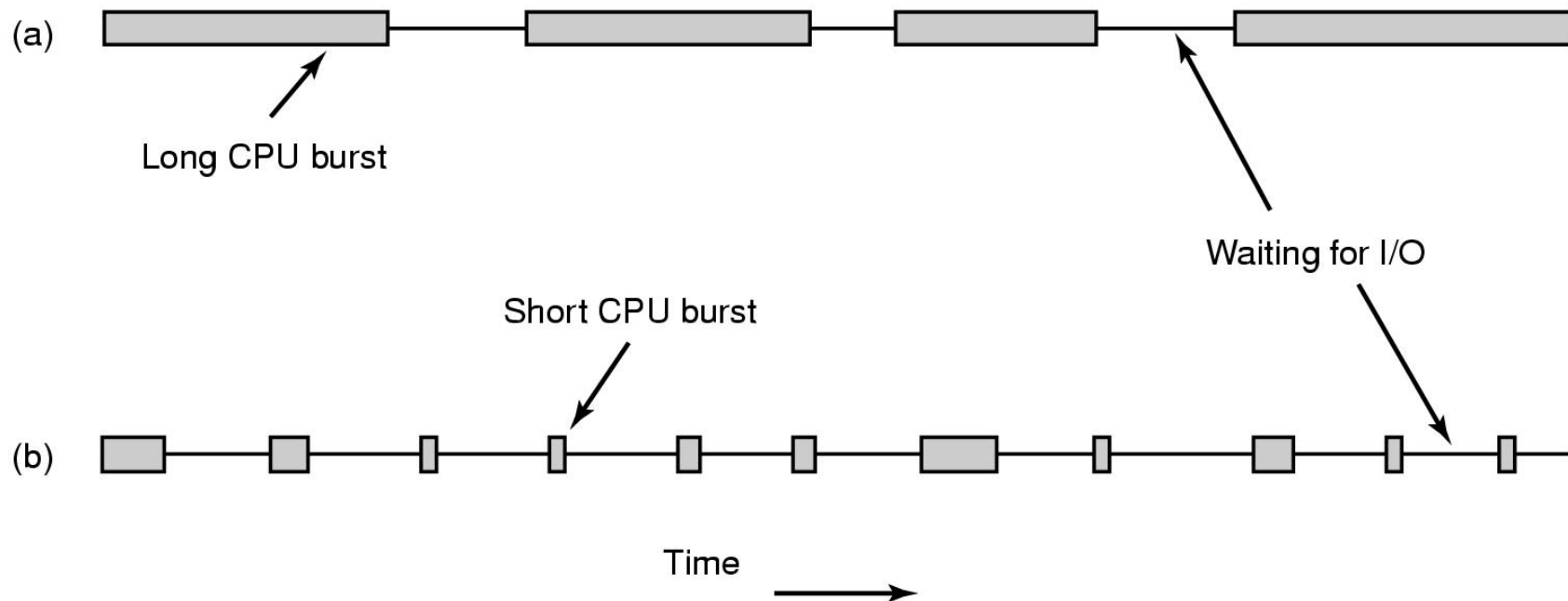
- Prioriza processos importantes
- Não deixa processos esperando indefinidamente



Processos CPU-bound e I/O-bound

- Há processos que permanecem a maior parte do tempo em execução, enquanto outros mais esperam por sinalização de periféricos
 - (a) Processo CPU-bound
 - (b) Processo I/O-bound

Escalonador deve considerar tais características !!!



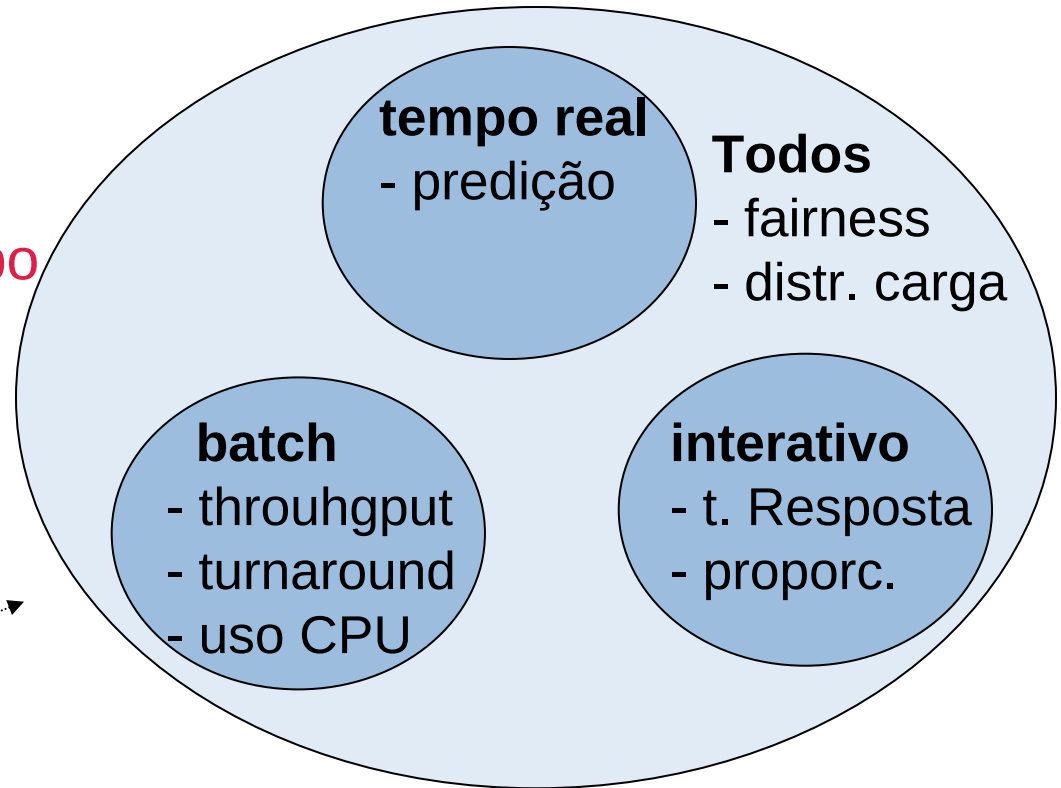
Categorias de Escalonadores

- Escalonadores específicos para os sistemas
 - **Batch** → preempção não é necessária
 - **Interativo** → preempção é necessária
 - **Tempo real** → processos são normalmente limitados por definição

Preempção:

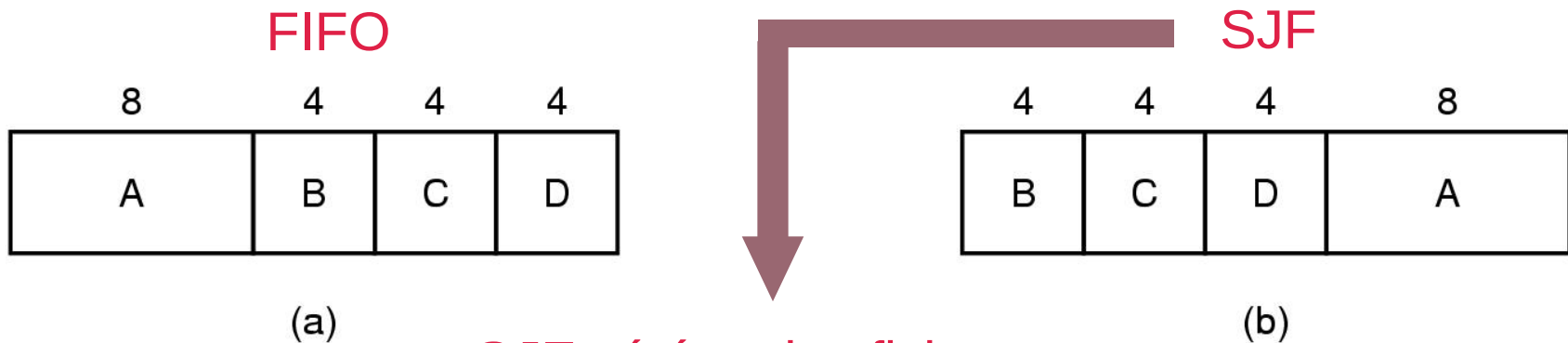
possibilidade de um processo ser retirado de execução pelo escalonador por limite de tempo

Principal objetivo de cada escalonador



O Escalonador Shortest Job First (SJF)

- O processo é selecionado pelo tamanho, o menor primeiro
- Adequado para sistemas *batch*
- Não preemptivo
- Tempo de execução dos processos deve ser conhecido a priori



SJF só é mais eficiente se todos os processos estiverem disponíveis simultaneamente

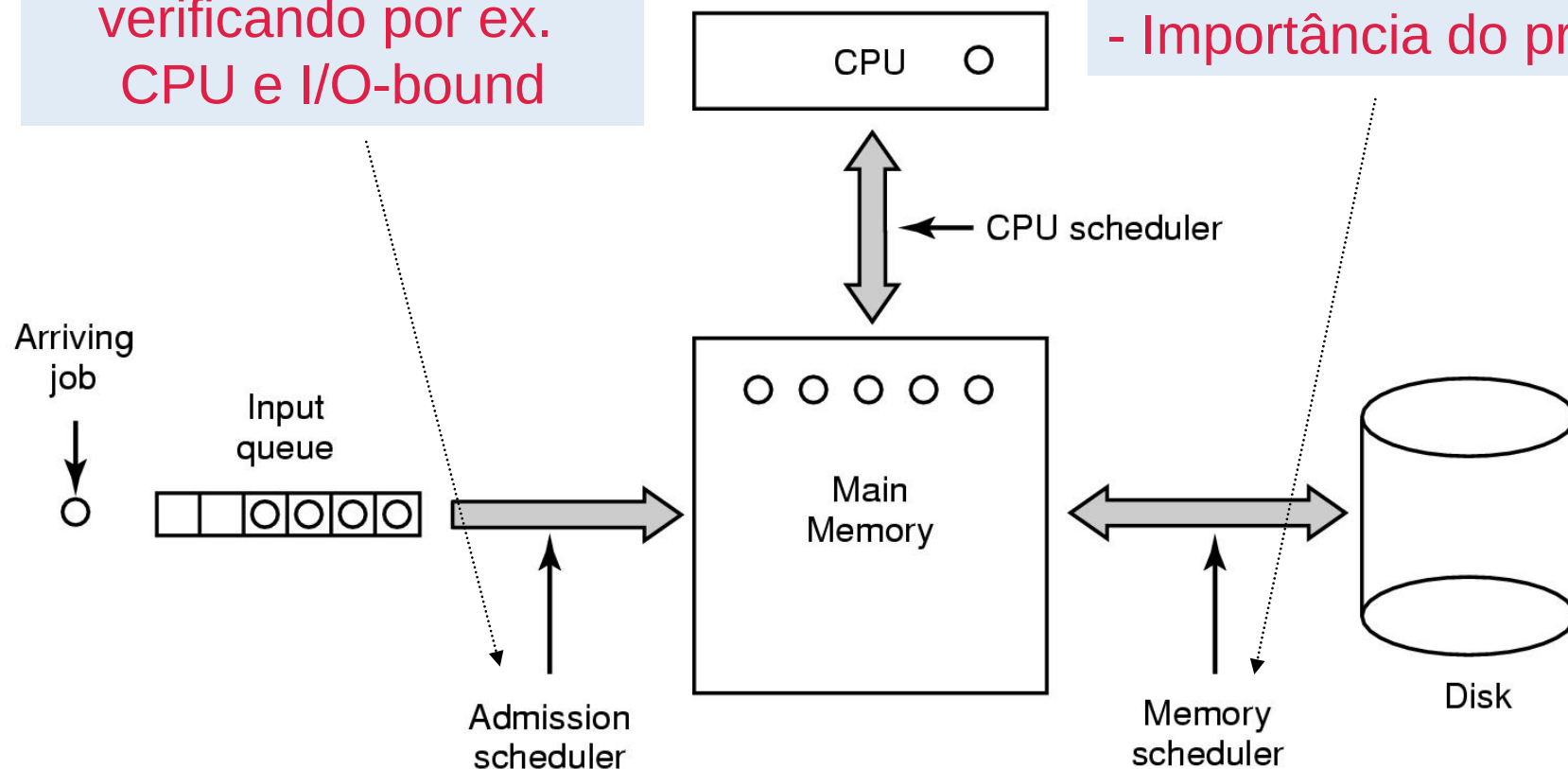
O Escalonador de 3 níveis

- Possui três escalonadores distintos:
 - de admissão, de memória e de CPU

Seleciona os processos verificando por ex. CPU e I/O-bound

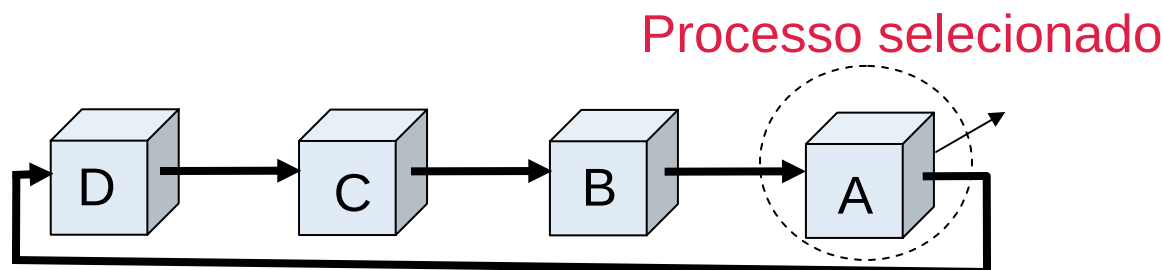
periodicamente verifica os processos

- Duração no disco
- Tempo de uso da CPU
- Tamanho do processo
- Importância do processo



O Escalonador Circular (Round-Robin)

- Um dos mais simples, antigo, justo (*fair*) e usado algoritmo
- A cada processo é atribuído uma fatia de tempo (**quantum**)
- Há uma fila com os processos prontos para execução
- Processos que usam a CPU além da sua fatia são colocados no final da fila
- Definição do valor do quantum é crítico
 - Se muito pequeno, ocorre perda de rendimento devido chaveamento de contexto
 - Se muito grande, torna operações interativas muito lentas



O Escalonador com prioridades

- Cada processo tem uma prioridade
- O de maior prioridade é escolhido em primeiro lugar
- Normalmente processos têm suas prioridades reduzidas a cada execução beneficiando processos de menor prioridade
- Podem ser agrupados em classes de prioridade

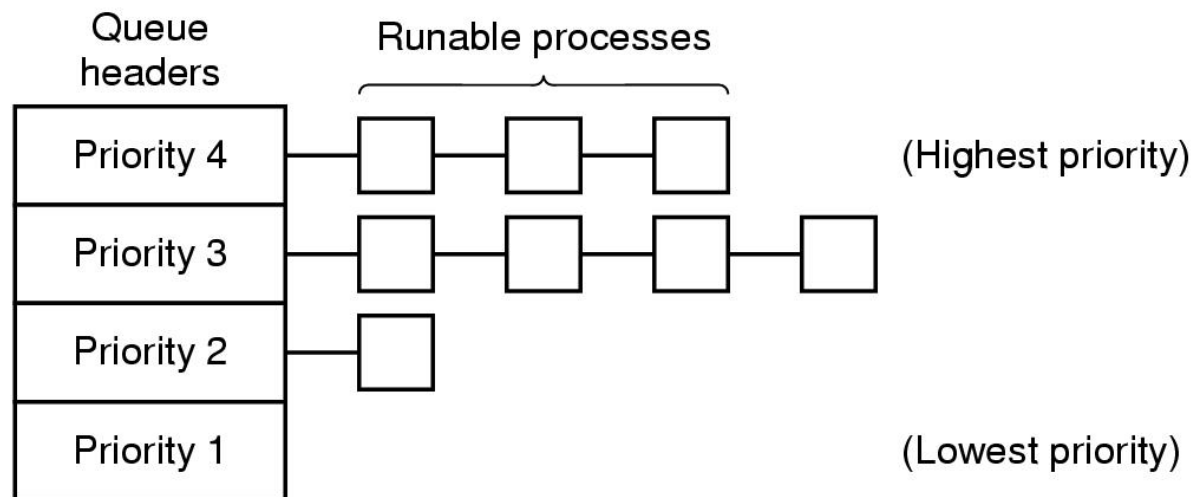
Como ajustar prioridades ?

Exemplo:

Fatia de tempo: 50 ms

1- Uso da fatia: 1ms
Prioridade: $50/1=50$

2- Uso da fatia: 25ms
Prioridade: $50/25=2$



Leitura Complementar

- ***Sistemas Operacionais Modernos, 2ª edição***
Andrew Tanenbaum, Editora Pearson, 2003.
- **Operating Systems (4th edition),**
William Stallings, Prentice Hall, 2001
- ***Sistemas Operacionais - Conceitos e Aplicações,***
A. Silberschatz, P. Galvin, e G. Gane, Editora Campus, 2001.
- **Sistemas Operacionais**
R. S. Oliveira, A. Carissimi e S. Toscani, Editora Sagra Luzzatto, 2004